Université des Sciences et de la Technologie d'Oran – Mohamed Boudiaf
Faculté des Mathématiques et Informatique
Département d'Informatique
Master I – Bases de données avancées

TP N° 2

"PL/SQL et triggers – EL_SAHA – الصحة"

Objectifs

Écrire, tester et diagnostiquer des blocs PL/SQL, des curseurs, des procédures, des fonctions et des triggers sur la base EL_SAHA : contrôles de données, effets automatiques entre tables et journal d'audit.

Prérequis	TP1 fait ; base EL_SAHA chargée (scripts v2), dans son état initial
Compte	`ELSAHA_ADMIN`
Scripts	`03_exemples_TP2_PLSQL_TRIGGERS_EL_SAHA.sql` (guide), `04_travail_TP2_PLSQL_TRIGGERS_EL_SAHA.sql` (à compléter)
Document annexe	`Fiche_MCD_EL_SAHA` : tables, colonnes et règles de gestion
Questions	**OBL** : obligatoires ; **OPT** : facultatives

Attention

Exécuter les exemples du script 03 dans l'ordre. Les exercices qui modifient la base se terminent par `ROLLBACK`. Les triggers créés restent dans le schéma : pour repartir de zéro, exécuter `01` puis `02` (les triggers sont supprimés avec leurs tables) et supprimer les packages créés (`DROP PACKAGE`). Ne pas rejouer `02` seul avec des triggers actifs.

Outil de travail : SQL Developer

SQL Developer (SQL*Plus accepté). **F5** exécute tout le script ; un bloc, une procédure ou un trigger se termine par une ligne `/`. Activer les messages avec `SET SERVEROUTPUT ON SIZE 1000000` (puis F5) ou le menu Affichage, *Sortie DBMS*. Les valeurs saisies se donnent avec `DEFINE variable = valeur` puis `&variable`. `SHOW ERRORS` détaille les erreurs de compilation. **Créer un trigger avant de le tester** : `CREATE`, `ALTER` et `DROP` valident la transaction en cours.

1 Structure d'un bloc

```
DECLARE
  -- variables, constantes, curseurs, exceptions (facultatif)
BEGIN
  -- instructions (obligatoire)
EXCEPTION
  -- traitement des erreurs (facultatif)
END;
/
```

`BEGIN ... END;` est obligatoire ; chaque instruction se termine par `;` ; la ligne `/` lance l'exécution du bloc.

2 Variables et types

```
DECLARE
  c_base     CONSTANT VARCHAR2(20) := 'EL_SAHA';
  v_nom      patient.nom%TYPE;        -- type d'une colonne
  v_patient  patient%ROWTYPE;         -- une ligne entiere
BEGIN
  SELECT * INTO v_patient FROM patient WHERE id_patient = 1;
  DBMS_OUTPUT.PUT_LINE(c_base || ' : ' || v_patient.nom);
END;
/
```

Préfixes conseillés : `v_` variable, `c_` constante, `p_` paramètre, `e_` exception. Un `SELECT ... INTO` doit retourner **exactement une ligne** : zéro ligne lève `NO_DATA_FOUND`, plusieurs lèvent `TOO_MANY_ROWS`. Sous Oracle 10g, une séquence ne s'affecte pas directement (`v := seq.NEXTVAL` est refusé) : écrire `SELECT seq.NEXTVAL INTO v FROM dual;`.

3 Conditions et boucles

```
IF v_stock = 0 THEN
  DBMS_OUTPUT.PUT_LINE('Rupture');
ELSIF v_stock <= v_seuil THEN
  DBMS_OUTPUT.PUT_LINE('Stock critique');
ELSE
  DBMS_OUTPUT.PUT_LINE('Stock normal');
END IF;

FOR i IN 1..5 LOOP
  DBMS_OUTPUT.PUT_LINE(i || ' au carre = ' || i * i);
END LOOP;
```

Boucles : `FOR` (nombre connu), `WHILE` (condition testée avant), `LOOP ... EXIT WHEN` (sortie explicite). Pour tester un jour de la semaine sans dépendre de la langue, utiliser `TRUNC(d) - TRUNC(d,'IW')` (lundi = 0).

4 SQL et transactions

```
BEGIN
  SAVEPOINT avant_modification;
  UPDATE rendez_vous SET statut = 'CONFIRME' WHERE statut = 'PLANIFIE';
  DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' lignes modifiees');
  ROLLBACK TO avant_modification;
END;
/
```

`SQL%FOUND`, `SQL%NOTFOUND` et `SQL%ROWCOUNT` décrivent la dernière instruction SQL. **Pas de `COMMIT` dans une procédure métier** : l'appelant valide ou annule.

5 Exceptions

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Introuvable');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE(SQLCODE || ' ' || SQLERRM);
...
RAISE_APPLICATION_ERROR(-20010, 'La date du rendez-vous est passee.');
```

Les codes personnalisés sont compris entre -20000 et -20999. `WHEN OTHERS` ne doit pas masquer une erreur : afficher `SQLCODE` et `SQLERRM`, puis relancer si la règle métier l'exige.

6 Curseurs

```
-- Boucle FOR : ouverture, lecture et fermeture automatiques
FOR r IN (SELECT nom, prenom FROM patient) LOOP
  DBMS_OUTPUT.PUT_LINE(r.nom || ' ' || r.prenom);
END LOOP;

-- Curseur explicite parametre
CURSOR c_rdv(p_id medecin.id_medecin%TYPE) IS
  SELECT id_rdv FROM rendez_vous WHERE id_medecin = p_id;

-- OPEN / FETCH / CLOSE
OPEN c_rdv(1);
LOOP
  FETCH c_rdv INTO v_id;
  EXIT WHEN c_rdv%NOTFOUND;
END LOOP;
CLOSE c_rdv;
```

Le curseur *implicite* (SQL) est géré par Oracle pour chaque instruction SQL ; le curseur *explicite* est déclaré et parcouru par le programmeur. Attributs : %FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN.

7 Procédures et fonctions

```
CREATE OR REPLACE FUNCTION f_total_paye_facture(p_id_facture IN facture.id_facture%TYPE)
  RETURN NUMBER IS
  v_total NUMBER;
BEGIN
  SELECT NVL(SUM(montant), 0) INTO v_total FROM paiement WHERE id_facture = p_id_facture;
  RETURN v_total;
END;
/
SHOW ERRORS FUNCTION f_total_paye_facture
SELECT id_facture, f_total_paye_facture(id_facture) FROM facture;
```

Modes des paramètres : IN (lecture seule), OUT (valeur retournée), IN OUT. Une *fonction* retourne une valeur avec RETURN et peut s'utiliser dans un SELECT ; une *procédure* s'appelle comme une instruction. Une fonction appelée depuis SQL évite les effets de bord et les transactions. Diagnostic : SHOW ERRORS, USER_ERRORS, USER_OBJECTS (colonne STATUS).

8 Triggers : principe

Un *trigger* est un bloc PL/SQL stocké, exécuté automatiquement par un INSERT, un UPDATE ou un DELETE sur une table.

- **Moment** : BEFORE (contrôler ou corriger les valeurs avant l'écriture) ou AFTER (réagir une fois la ligne écrite, par exemple alimenter une autre table).
- **Niveau** : FOR EACH ROW (une exécution par ligne ; :NEW et :OLD disponibles) ou instruction (une exécution par ordre SQL).
- **Événement** : INSERTING, UPDATING, DELETING distinguent le cas dans un trigger multi-événements ; UPDATE OF colonne limite le déclenchement ; WHEN (NEW.x IS NOT NULL) filtre les lignes (sans deux-points).

Événement	:OLD	:NEW
INSERT	vide	valeurs insérées
UPDATE	valeurs avant	valeurs après
DELETE	valeurs supprimées	vide

8.1 Syntaxe

```
CREATE OR REPLACE TRIGGER trg_exemple
BEFORE INSERT OR UPDATE OF cout_acte ON acte
FOR EACH ROW
WHEN (NEW.cout_acte IS NOT NULL)
BEGIN
  IF :NEW.cout_acte < 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Cout negatif interdit');
  END IF;
END;
/
SHOW ERRORS TRIGGER trg_exemple
```

Dans un trigger BEFORE ligne, on modifie :NEW (:NEW.nom := UPPER(:NEW.nom)). Dans un trigger AFTER, on agit sur d'autres tables. Une erreur levée par RAISE_APPLICATION_ERROR (codes de -20000 à -20999) annule l'instruction qui a déclenché le trigger.

8.2 Règles à respecter

- **Pas de COMMIT, ROLLBACK ni SAVEPOINT** dans un trigger (ORA-04092) : la transaction appartient à l'appelant.
- **Table mutante** (ORA-04091) : un trigger ligne ne peut ni lire ni modifier la table qui est en cours de modification. Lire une *autre* table est permis. Sous Oracle 10g, le contournement classique est un package qui mémorise les lignes concernées (trigger ligne) et les traite après l'instruction (trigger instruction), voir l'exercice 25.
- **Transaction** : CREATE, ALTER et DROP TRIGGER sont des instructions DDL qui valident la transaction. Créer, *puis* tester, *puis* ROLLBACK.
- **Ordre** : plusieurs triggers du même moment sur la même table s'exécutent dans un ordre non garanti en 10g ; ne pas faire dépendre un trigger d'un autre au même moment.
- **Redondance** : une contrainte (CHECK, UNIQUE, clé étrangère) vaut mieux qu'un trigger quand elle suffit ; le trigger sert aux règles qu'elle ne sait pas exprimer.

8.3 Gérer les triggers

```
SELECT trigger_name, table_name, triggering_event, trigger_type, status
FROM user_triggers ORDER BY table_name, trigger_name;
SELECT name, line, text FROM user_errors WHERE type = 'TRIGGER';
ALTER TRIGGER trg_exemple DISABLE;      -- ou ENABLE
DROP TRIGGER trg_exemple;
```

9 Méthode de test

Pour chaque programme ou trigger : un cas normal (ou accepté), une valeur limite, un identifiant inexistant, un cas interdit (ou refusé, avec son message) ; vérifier les données avant et après, y compris dans les autres tables touchées par un trigger ; ROLLBACK pour les essais ; consulter SHOW ERRORS ; commenter le résultat. Un cas refusé se teste dans un bloc BEGIN ... EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE(SQLERRM); END;.

10 Exercices

Les questions **OBL** sont indispensables ; les questions **OPT** sont facultatives. Travailler dans 04_travail_TP2_PLSQL_TRIGGERS_EL_SAHA.sql : énoncé en commentaire, code, deux tests au moins, résultat.

10.1 A. Blocs, variables, conditions et exceptions

Tester chaque bloc avec une valeur normale, une valeur limite et une valeur interdite. Les valeurs saisies se donnent avec DEFINE.

1. **[OBL]** Écrire la fonction F_FACTORIEL(n) (erreur -20001 si n est négatif ou non entier), afficher F_FACTORIEL(10), puis le factoriel d'une valeur saisie avec DEFINE.

2. **[OBL]** Écrire un bloc qui reçoit l'identifiant d'un patient et affiche son nom, son prénom, son groupe sanguin et sa ville. Traiter le patient inexistant.

3. **[OBL]** Calculer l'âge d'un patient en années entières (`MONTHS_BETWEEN`, sans diviser par 365) et afficher sa catégorie : mineur (moins de 18 ans), adulte (18 à 59 ans) ou senior (60 ans et plus).

4. **[OBL]** Rechercher un patient avec `SELECT ... INTO`, d'abord par son nom (essayer un nom absent puis un nom existant), puis par sa ville. Expliquer `NO_DATA_FOUND` et `TOO_MANY_ROWS`.

5. **[OPT]** Vérifier si une date tombe un vendredi ou un samedi, sans dépendre de la langue de la session (`TRUNC(d) - TRUNC(d,'IW')` : lundi vaut 0, vendredi 4, samedi 5).

10.2 B. Curseurs

6. **[OBL]** À l'aide d'une boucle `FOR` sur une requête, afficher chaque patient avec son nombre de consultations, y compris les patients sans consultation.

7. **[OBL]** Créer un curseur paramétré qui affiche les consultations des médecins d'un service entre deux dates (paramètres : service, date de début, date de fin).

8. **[OPT]** Parcourir à la main (`OPEN`, `FETCH`, `CLOSE`) les médicaments dont le stock est sous le seuil d'alerte et afficher leur nombre avec `%ROWCOUNT`.

10.3 C. Procédures, fonctions et transactions

Les exercices qui modifient la base se terminent par `ROLLBACK` : la base doit revenir à son état initial. Aucun `COMMIT` dans les procédures.

9. **[OBL]** Créer `P_RESUME_PATIENT(p_id_patient)` qui affiche l'identité du patient, son nombre de rendez-vous, de consultations et d'hospitalisations ; traiter le patient inexistant.

10. **[OBL]** Créer `F_RESTE_FACTURE(p_id_facture)` qui retourne le montant restant à payer, c'est-à-dire le montant total diminué de la remise (en pourcentage) puis du total déjà payé. Lever l'erreur `-20101` si la facture n'existe pas. Vérifier avec les factures 2 et 20.

11. **[OBL]** Créer `P_AJOUTER_PATIENT` (nom, prénom, date de naissance, sexe, adresse électronique, numéro de sécurité sociale, identifiant en sortie) : utiliser `SEQ_PATIENT`, refuser une adresse électronique ou un numéro déjà utilisés (erreurs `-20111` et `-20112`). Tester un ajout, un doublon, puis `ROLLBACK`.

12. **[OBL]** Modifier le stock de plusieurs médicaments dans un même bloc, provoquer volontairement une erreur (stock négatif), puis annuler seulement la partie incorrecte avec un `SAVEPOINT`. Vérifier les stocks avant, pendant et après, puis `ROLLBACK`.

10.4 D. Triggers

Créer chaque trigger *avant* ses tests (la création valide la transaction), tester dans une transaction, puis `ROLLBACK`. Les règles de gestion à faire respecter sont celles de la fiche annexe. Ne pas rejouer le script 02 avec des triggers actifs : repartir de 01 puis 02.

13. **[OBL]** Créer `TRG_PATIENT_BIU` (avant insertion ou modification, niveau ligne) : nom en majuscules, prénom avec une majuscule initiale, adresse électronique en minuscules, sans espaces superflus. Tester avec un patient saisi en désordre, puis `ROLLBACK`.

14. **[OBL]** Créer `TRG_RDV_BIU` (avant insertion ou modification, niveau ligne) : refuser une date passée à l'insertion ou lorsque la date change (`-20201`), une heure qui n'est pas de la forme `HH:MM` valide (`-20202`), le changement de statut d'un rendez-vous clos `ANNULE`, `REALISE` ou `ABSENT` (`-20203`) et le retour de `CONFIRME` à `PLANIFIE` (`-20204`). Une modification qui ne change pas la date ne doit pas être refusée.

15. **[OBL]** La consultation répète le patient et le médecin du rendez-vous. Créer `TRG_CONSULT_BI` (avant insertion, seulement si `ID_RDV` est renseigné, clause `WHEN`) : refuser un patient ou un médecin différents de ceux du rendez-vous (`-20401`), un rendez-vous `ANNULE` (`-20402`) et un rendez-vous inexistant (`-20403`).

16. **[OBL]** Créer `TRG_CONSULT_AI` (après insertion d'une consultation liée à un rendez-vous, niveau ligne) qui passe le rendez-vous à `REALISE` s'il était `PLANIFIE` ou `CONFIRME`. Tester avec le rendez-vous 81 (patient 17, médecin 6), puis `ROLLBACK`.

17. **[OBL]** Créer `TRG_PRESCRIPTION_STOCK` (avant insertion, modification ou suppression d'une prescription, niveau ligne) qui diminue le stock du médicament de la quantité prescrite, le rétablit en cas de suppression, gère le changement de médicament ou de quantité, et refuse un stock insuffisant (`-20701`). Tester avec les médicaments 1 (stock 250) et 3 (stock 6), puis `ROLLBACK`.

18. **[OBL]** Créer `TRG_FACTURE_STATUT_BU` (avant modification du statut, niveau ligne) : une facture `PAYEE` ou `ANNULEE` ne change plus de statut (`-20801`) et une facture `PARTIELLE` ne revient pas à `EMISE` (`-20802`). Tester avec les factures 1 (payée), 2 (partielle) et 3 (émise).

19. **[OBL]** Créer `TRG_FACTURE_AUDIT` (après insertion, modification ou suppression, niveau ligne) qui écrit dans `JOURNAL_OPERATION` le type d'opération, la table, la clé, et les anciennes et nouvelles valeurs (statut, montant, remise). L'utilisateur et la date sont fournis par les valeurs par défaut de la table. Tester, lire le journal, puis `ROLLBACK`.

20. **[OBL]** Interroger `USER_TRIGGERS` (nom, table, événement, type, statut) et `USER_ERRORS`. Désactiver `TRG_FACTURE_STATUT_BU`, constater qu'une facture payée peut alors changer de statut, annuler cette modification puis réactiver le trigger. Pourquoi faut-il désactiver *avant* de modifier ?

21. **[OPT]** Créer `TRG_HOSPIT_BIU` (avant insertion ou changement de chambre) : refuser une chambre dont `DISPONIBLE` vaut `'N'` (`-20301`) ou inexistante (`-20302`), et remplir `COUT_JOURNALIER` avec le tarif de la chambre s'il est absent. Tester avec les chambres 1 et 19.

22. **[OPT]** Créer `TRG_SERVICE_CHEF` (avant insertion ou modification du chef, `WHEN`) : le médecin chef doit être affecté au service qu'il dirige (`-20501`) et exister (`-20502`). Tester avec les médecins 2 (service 1) et 3 (service 2) pour le service 1.

23. **[OPT]** Créer `TRG_PRESCRIPTION_PEREMPTION` (avant insertion) qui refuse de prescrire un médicament dont la date de péremption est dépassée (`-20901`). Aucun médicament n'est périmé dans la base : en périmer un par `UPDATE` pour le test, puis `ROLLBACK`.

24. **[OPT]** Créer `TRG_PATIENT_AUDIT` (après modification d'un patient) qui journalise seulement les *noms des colonnes modifiées* (nom, prénom, adresse, ville, téléphone, e-mail, groupe sanguin, antécédents, allergies), sans écrire de valeur médicale dans le journal. Pourquoi ?

25. **[OPT]** (a) Créer un trigger `AFTER INSERT` niveau ligne sur `PAIEMENT` qui lit la table `PAIEMENT` (somme des paiements d'une facture) : provoquer et expliquer l'erreur `ORA-04091`, puis supprimer ce trigger. (b) Corriger avec un package qui mémorise les factures concernées (trigger ligne) et les traite après l'instruction (trigger instruction) : le statut passe à `PARTIELLE` ou `PAYEE`, et un total supérieur au montant net est refusé (`-20951`). Tester avec les factures 3 (émise, net 8 500) et 2 (partielle, reste 3 000).

11 Travail à rendre

- `tp2_plsql_nom_binome.sql` : solutions et tests commentés ;
- `tp2_plsql_nom_binome.txt` produit avec `SPOOL` (tests réussis, refusés et erreurs) ;
- la liste des procédures et fonctions (`USER_OBJECTS`) et celle des triggers (`USER_TRIGGERS`) avec leur statut ;
- une phrase pour chacune de ces différences : curseur implicite et explicite, procédure et fonction, `BEFORE` et `AFTER` ; et une phrase expliquant `ORA-04091` ;
- la base sans essai conservé (comptages du TP1).

Résultat attendu

À la fin du TP, l'étudiant écrit, teste et diagnostique des blocs, des curseurs, des procédures, des fonctions et des triggers sur EL_SAHA, et sait quand préférer une contrainte, une procédure ou un trigger.